```
        get => _indexerList[i];                                    ❸
        set => _indexerList[i] = value;                            ❸
    }
    public List<SubClass> NullList { get; }                        ❹
    public Dictionary<int, SubClass> IntegerKeys { get; set; }     ❹
    public IEnumerable<SubClass> ReadOnlyEnumerable { get; }       ❹
        = new List<SubClass>();                                    ❹

    public SubClass _setOnly = null;                               ❺
    private readonly List<SubClass> _indexerList                   ❺
        = new List<SubClass>();                                    ❺

    public class SubClass
    {
        public string Value { get; set; }
    }
}
```

❶ The binder can bind simple and complex object types, and read-only properties with a default.

❷ The binder will also bind collections, including interfaces.

❸ The binder can't bind nonpublic, set-only, null-read-only, or indexer properties.

❹ These collection properties can't be bound.

❺ The backing fields for implementing SetOnly and Indexer properties—not bound directly

As shown in the listing, the binder generally supports collections—both implementations and interfaces. If the collection property is already initialized, the binder uses the initialized value; otherwise, the binder may be able to create the collection instance automatically. If your property implements any of the following interfaces, the binder creates a `List<>` of the appropriate type as the backing object:

- `IReadOnlyList<>`
- `IReadOnlyCollection<>`
- `ICollection<>`
- `IEnumerable<>`

WARNING You can't bind to an `IEnumerable<>` property that has already been initialized, as this interface doesn't expose an `Add` function, and the binder